CCIThink: Beginner Code Walkthrough and Defense Guide

Read the code, rebuild it in stages, and explain the measured results

Boncajes, Rhian Jared D.

Marcos, Maricar N.

Carandang, Jamie Pearl U.

CCIT · FAITH Colleges · November 2026

1. Start with the project in plain words

CCIThink is a Flask website that answers public department questions using 22 source FAQs. The browser collects a question, Python selects supported source records, and the browser displays a short answer with bold key facts. A separate local machine-learning model predicts the likely FAQ label. A separate analysis page explains the dataset, charts and measured experiment.

Learn the core request path first: app.py, conversation.py, retriever.py and answers.py. Next study intent_model.py, dataset.py, evaluation.py and analyze.py. Study optional AI troubleshooting and document-export helpers after that. The source-function index and complete file inventory at the end let you find every part without treating generated reports or images as application logic.

The backend was made easier to read by expanding compact expressions into named variables, ordinary loops and explicit if/else branches. Numbered comments show the chat and experiment sequence. These changes preserve the algorithm, input/privacy checks and UI behavior; the same regression suite verifies them.

Three different things that must not be confused

Part	What it does
Source FAQ records	Supply department facts. A model prediction does not certify that those facts are current.
Local Naive Bayes classifier	Learns question-to-FAQ label patterns from the constructed training CSV; works without an API key.
Optional Gemini/OpenAI generator	Phrases an answer from retrieved records; needs a private provider key and working model access.

2. The Python and web ideas you need

Vocabulary you should be able to explain

Term / syntax	Meaning in this project
Variable	A name for a value, such as message or api_key.
List: []	An ordered collection, such as selected FAQ records.
Dictionary: {}	Named fields, such as a record with id, question and answer.
Set: set() or {values}	Unique values, useful for vocabulary and program IDs.

Term / syntax	Meaning in this project
Counter	A dictionary-like standard-library tool for counting words or labels.
for / if / return	Visit items; make a decision; leave a function with its result.
Function	A named step with inputs and an output, such as retrieve(question, records).
Class / self	A model object groups learned counts and methods; self refers to that model instance.
@classmethod	Allows IntentModel.fit(...) to build a model object from training rows.
@app.post(...)	A Flask decorator connects a URL and HTTP method to a Python function.
JSON	Text representing lists/dictionaries. The browser and Flask exchange JSON.
GET / POST	GET reads a page/resource; POST sends question data to a route.
try / except	Handle an expected failure with a useful response instead of a server crash.
with	Closes a file or client automatically when the block finishes.
Path / ROOT	Build portable file paths relative to the repository, including on Windows.
A & B / A - B	Set intersection finds shared words; subtraction removes words.
lambda	A short function used here mainly to describe sorting order.
**dictionary	Copy dictionary fields into a new dictionary; it is unrelated to bold text.
Regular expression	A text pattern. For example, a word boundary helps match BSCS as a word.
Hash / fingerprint	A compact identifier for file/content changes, used for traceability and stale-model checks.

```
question = "Who teaches BSCS?"
record = {"id": "FAQ-018", "question": question}

# A list keeps several records in order.
records = [record]
for item in records:
    print(item["id"])
```

3. Trace one real question from start to finish

Use "Do you guys offer BSCS?" as a concrete example. The following is the real application flow, not a claim that the language model knows department facts by itself.

The request and response path

Step	File and action
1. Type and send	templates/index.html has the textarea; static/js/script.js reads its text and creates a JSON request.
2. Validate	app.py chat() accepts a nonempty string up to 1,000 characters and checks context/mode.
3. Protect private records	conversation.py detects requests for private grades/passwords; this question is public.
4. Resolve context	The explicit BSCS name becomes the selected program; a follow-up can use it.
5. Select sources	retriever.py recognizes the offering topic and BSCS, checks source support and ranks eligible records.
6. Add a learned hint	intent_model.py supplies relative FAQ scores; at most 0.25 times a score is added after eligibility checks.
7. Build the answer	In FAQ mode, answers.py extracts the BSCS offering from FAQ-006 and marks the degree name with double asterisks.
8. Return JSON	app.py sends reply, mode, allowed context and backend metadata.
9. Display safely	script.js builds text nodes and a strong element, so the key fact is bold without executing answer HTML.

```
// Browser sends this JSON to POST /api/chat:
{
  "message": "Do you guys offer BSCS?",
  "context": {"programs": []}
}
```

The visible FAQ response is "Yes. CCIT offers Bachelor of Science in Computer Science (BSCS)." The degree name is bold in the page. Source IDs, verification notes and original-source details stay in the backend/API rather than the answer bubble. The small FAQ label accurately identifies retrieval rather than AI generation.

Try two other traces. "What time do the labs open?" fails the unsupported-detail check because lab names do not establish opening hours. "Show my student grades" is handled before retrieval by the privacy check. A strong classifier score cannot bypass either check.

4. The main backend files

app.py: the traffic controller

The imports bring in Flask, JSON, environment settings and the smaller project modules. ROOT points to the repository directory. load_dotenv reads the private .env beside app.py. TOPICS holds the three existing home-card groups; it is UI configuration, not a training dataset.

create_app() creates a Flask object, reads configuration, loads knowledge.json and tries to load model.json. It then connects the URLs to route functions. Keeping this setup in a function allows tests to make a separate app with synthetic settings without editing the real .env. The last two lines run the app only when this file is executed directly.

Routes to know

Route	Purpose
GET /	index() renders the preserved chat HTML.
GET /api/health	health() tells the browser whether source records and the chosen answer mode are available. It never returns an API key.
GET /api/suggestions	suggestions() groups reviewed source questions into the home topic cards.
POST /api/chat	chat() follows the nine numbered input-to-answer steps.
HTTP error handler	Returns consistent JSON for ordinary HTTP errors, such as a request that is too large.

Read the nine numbered blocks in chat() aloud. Each early return ends a case immediately: invalid input, privacy, greeting, setup failure, clarification, missing source or provider failure. This avoids hiding the main path inside deeply nested conditionals. Configuration values live in app.config so routes use the same loaded source/model settings.

conversation.py: small follow-ups, not a conversation database

read_context() accepts only a list of up to three known program IDs and an optional pending topic. It rejects arbitrary facts/instructions. resolve_question() notices explicit program names, references such as "this program", and short follow-ups such as "And faculty?". It either appends the previous program name to the question or asks which program is meant. is_private_question() checks for a sensitive term together with a personal reference.

```
# Example context passed by the browser:
context = {"programs": ["bscs"]}

# "What jobs after this program?" can then be resolved to BSCS.
# With no program context, the app asks the student to choose.
```

A refresh or New chat clears the frontend context. The server does not keep one shared conversation for all users. Every request carries its own small validated context. This is why a follow-up can work without a database or login system.

retriever.py: choose sources before writing an answer

The tables at the top contain program IDs, phrase rules, word aliases, ignored filler words and topic keywords. These are readable matching rules, not learned Naive Bayes weights. For example, jobs and careers are treated consistently; a college name containing Information Technology is not automatically evidence about the BSIT staff.

Retriever helpers

Function	Why it exists
normalize	Standardizes phrases, Unicode/case and program wording. Uppercase IT can mean the program while lowercase it can be a pronoun.
_canonical_word	Maps aliases and close long intent-word typos. A small lru_cache avoids repeating word-normalization work. It does not store conversations.
tokens	Produces a set of meaningful normalized words after skipping filler words.
program_entities	Keeps only BSIT, BSCS or BSEMC from the token set.
intents	Finds the requested topics, with rules to distinguish offerings, definitions, DOST and other overlapping categories.
_supported_detail	Rejects a record if the question needs an unsupported fee, date, contact, location, hours or other missing detail.
retrieve	Filters, scores, sorts and selects source records, with comparison/multiple-topic handling.

Inside retrieve(), read the five numbered comments. First recognize the words. Next filter eligible sources. Then count document word frequency. Score each eligible candidate for each requested topic. Finally select the appropriate records and avoid duplicate IDs. Coverage must be at least 0.6; title/keyword matches receive extra weight; rarer matched words receive a log-based weight. Exact source questions get an additional bonus. The learned model adds only the fixed MODEL_RANK_BONUS of 0.25 times its score.

There are two separate reasons for ranking: lexical matching checks actual question/source word support; learned scores help distinguish likely FAQ intents. Neither creates facts. A record still needs to be reviewed and eligible. The rules preserve unsupported-question fallback even if a closed-set model predicts some label.

answers.py: shorten and format the selected facts

_sentences() splits text for short summaries. _summary() chooses a source-based presentation by category: program lists, scholarships, career/faculty bullets, laboratory names or a short definition. _emphasize() marks important phrases already present in that reply using double asterisks. format_faq() combines selected records. source_metadata() prepares review/currency information separately from visible text. This is formatting/extraction, not model training.

Department answers remain in knowledge.json. The formatter does not invent new tuition amounts, contact details or qualifications. If the source is edited, review the summary and bolding behavior as well as the factual content. Source review metadata is still retained for maintenance even though the chat bubble is kept clean.

ai_service.py and errors.py: optional external AI

generate() packages the resolved question and selected records as JSON, then uses the configured official Gemini or OpenAI SDK. SYSTEM asks for concise source-only wording and bold key facts. Each provider call has a bounded timeout and no automatic retry loop. list_models() asks the provider which model IDs it returns; listing is not proof of generation access or free quota.

errors.py contains EmptyResponseError, a simple AIProblem container, and diagnose(). AIProblem stores a fixed code, readable message, maintainer action and optional status. diagnose() recognizes structured provider statuses and error types, then returns a fixed diagnosis. Raw provider error bodies can include credentials, so they are not echoed or logged. app.py uses this diagnosis to show retry/fallback options.

Useful error meanings

Status / case	Meaning
400 from the app	The question/context request is invalid.
401/403 upstream	The provider rejected access or the API key/project permission.
404 upstream	The configured model is unavailable for that project.
429 upstream	Quota or rate limiting; another listed model does not guarantee free access.
502 from the app	The app reached its provider path but generation failed.

Status / case	Meaning
503 from the app	Required data/model/provider setup is unavailable for that route or mode.

check_ai.py: a troubleshooting utility

main() reads CLI options; run() prints safe configuration metadata. model_candidates() ranks compatible Gemini Flash/Flash-Lite text IDs. find_working_model() tries at most three and stops at first success or on quota/access/connection problems. It does not rewrite .env. These optional commands can use provider quota; ordinary FAQ mode and local training do not call them.

5. The local machine-learning code

What the model learns

The model receives question text and learns one of 22 FAQ IDs as the target. It does not learn student grades, scholarship decisions or whether a factual statement is current. Each class has eight training questions. A vocabulary contains the training words and, for one configuration, adjacent word pairs. Class counts and word counts are enough to construct this multinomial Naive Bayes model.

intent_model.py step by step

Functions and methods

Part	Action
clean_question	NFKC, case, apostrophe and whitespace normalization.
features	Counts single words; if ngram_max is 2, also counts adjacent pairs.
knowledge_fingerprint	Hashes the source IDs/questions/answers to detect a changed knowledge file.
IntentModel.__init__	Checks artifact structure, keeps counts, and precomputes priors/likelihood denominators.
IntentModel.fit	Loops through training rows, counts labels/terms and builds the vocabulary. Returns a model object.
IntentModel.predict	Counts known input features, calculates one log score per class and returns sorted relative scores.
IntentModel.save	Writes ordinary JSON counts and metadata rather than executable pickle objects.

Part	Action
load_model	Reads the artifact, checks the knowledge fingerprint/labels and returns None for a missing, stale or invalid model.

```
# Unigrams: individual words.
features("who teaches bscs", ngram_max=1)
# Counts include: who, teaches, bscs

# Bigrams: adjacent pairs, in addition to the individual words.
features("who teaches bscs", ngram_max=2)
# Also includes: "who teaches", "teaches bscs"
```

fit() is training: it changes the learned counts based on labeled examples. predict() is inference: it uses saved counts without learning from the current user question. Training happens in analyze.py, not on each chat request. The browser cannot submit a new training dataset to a public retraining endpoint.

```
# The main prediction calculation, written as ordinary loops:
word_score = 0.0
for term, count in known.items():
    likelihood = (terms.get(term, 0) + alpha) / denominator
    word_score += count * math.log(likelihood)

class_score = log_prior + word_score
```

The prior is the fraction of training rows belonging to a class. The likelihood is the smoothed fraction of that class's feature counts assigned to a term. Alpha prevents zero likelihoods; alpha 1 is Laplace smoothing and alpha 0.5 is Lidstone smoothing. Logs turn many multiplications into additions and avoid very tiny products. Subtracting the largest log score before exponentiating keeps normalization stable. Relative scores add to one, but they are not calibrated probabilities that an answer is correct.

A tiny hand calculation you can reproduce

This is a learning example, not the department experiment. Suppose class A has the text "apple apple" and class B has "pear". Each has one training row, so both priors are 1/2. The vocabulary has two words. With alpha = 1, the apple likelihood is (2 + 1)/(2 + 2) = 3/4 in A and (0 + 1)/(1 + 2) = 1/3 in B. For input "apple", normalized A weight is (3/4)/[(3/4) + (1/3)] = 9/13, about 0.6923. The matching unit test verifies exactly this arithmetic.

The naive assumption is that features are conditionally independent given a class. Real words are related, so this assumption is imperfect. Bigrams add some local order but do not make the model understand language like a person. With no known input features, predict() returns an empty list instead of choosing an arbitrary equal-prior class.

dataset.py: honest data preparation

FIELDS, SPLITS and LANGUAGES define the allowed CSV structure. count_values() creates descriptive counts. load_dataset() reads rows, checks required values, normalizes text/labels and retains valid questions. It counts removed missing/invalid/within-split duplicate rows, but stops on conflicting labels or exact token sequences shared across splits. A repeated question must not appear in training and testing.

The prepared dataset has 264 rows, all normalized. There were zero missing, duplicate or invalid rows to remove; seven longer valid questions were retained. This is a constructed corpus, so reporting zero dirty rows is more accurate than pretending real-data cleaning problems occurred. Approximate language, split, provenance and sample IDs are metadata; only question text becomes input features. The FAQ ID is the separate target.

evaluation.py: count performance rather than guess

classification_metrics() loops through each label to count true positives, false positives, false negatives and support. It calculates precision, recall and F1, then averages across classes for macro metrics. Separate loops build the confusion matrix and total accuracy. evaluate() runs model predictions for the selected rows, keeps each result for audit, and adds a descriptive language-tag breakdown.

Metrics to explain

Metric	Meaning
Accuracy	Correct predictions divided by all evaluated questions.
Precision for one FAQ	Of questions predicted as this FAQ, how many truly belonged to it?
Recall for one FAQ	Of questions actually belonging to this FAQ, how many did the model find?
F1	The harmonic balance 2PR/(P+R); zero when the denominator is zero.
Macro F1	Average of the 22 individual class F1 scores; each FAQ has equal weight.
Confusion matrix	Rows are actual labels, columns predicted labels; the diagonal is correct predictions.

analyze.py: the five-step experiment

Read the five numbered comments in run(). First load the CSV and source labels. Then group rows using the fixed split column. Train four feature/smoothing configurations and evaluate them on validation only. Evaluate the selected winner and a majority baseline on test. Finally save the model, prepared rows, predictions, comparison, figures and JSON report.

better_configuration() applies the prespecified rules with explicit if statements: higher validation macro F1; then higher validation accuracy; then fewer n-grams; then alpha closest to 1. save_csv() keeps a consistent column order and LF newlines. figures() uses Matplotlib to draw class counts, token lengths, approximate language counts, the validation comparison and the selected-model test confusion matrix. main() reads command-line arguments and reports clear errors.

The fixed allocation is 176 train, 44 validation and 44 test rows. Training fits the vocabulary and counts. Validation chooses parameters. Testing estimates performance for the selected configuration. Looking at test errors and repeatedly changing parameters to improve those same results would weaken the evaluation; use fresh test evidence for future development.

The actual result to defend

Measured results, not participant survey results

Evidence	Actual interpretation
Classifier test: 28/44, 63.6% accuracy	A small constructed held-out question set; not general student accuracy.
Macro F1: 0.595	Several intents perform poorly even though overall accuracy exceeds the baseline.
Majority baseline: 2/44, 4.5%	Always predicts FAQ-001 after the alphabetical tie-break; a weak comparison.
English 16/22; Filipino 3/8; Taglish 9/14	Descriptive approximate-tag groups; too small for population-level language conclusions.
263 automated tests	Software behavior and arithmetic checks; not 263 users.
26 browser scenarios	Executed UI/recovery/safety checks; not a user-satisfaction study.
95/95 HTTP cases	Earlier development regression cases; separate from the 44 classifier test questions.

6. The separate research page

research/web.py uses a Flask Blueprint, which is just a group of related routes registered by app.py. read_report() reads the saved result JSON. analysis() renders templates/analysis.html. analysis_data() exposes the saved report as JSON. analysis_figure() serves only the five allowed chart filenames. classify() validates a public question, uses the already loaded local model and returns up to three labels with relative scores.

templates/analysis.html turns the saved dictionaries into tables, figures, limitations and the prediction form. Jinja escapes inserted text by default. analysis.css is scoped to the analysis page. analysis.js listens for form submission, sends JSON to /api/classify, handles errors/timeouts and creates text nodes for results. The full-size image links help read detailed charts on a small screen. This page is separate so the original chat home remains visually unchanged.

7. The frontend files

index.html and style.css

The HTML defines elements such as question, send, clear, messages, welcome, suggestions, dialogs and status. JavaScript finds these elements by their IDs. The CSS loads self-hosted Inter fonts, defines the red palette and sets the header, composer, cards and message layout. Media queries adapt the same controls for narrow screens; they do not change the backend algorithm.

script.js: the important functions

Frontend function map

Function(s)	Purpose
updateGreeting	Shows Guest or the optional tab-local preferred name.
resizeInput / setBusy	Adjusts textarea height and enables/disables controls while a request is pending.
scrollToLatest / beginConversation	Keeps the latest reply visible and switches from welcome cards to the conversation view.
renderEmphasis / renderText	Creates safe paragraphs/lists/strong elements from plain text and the supported bold markers.
placeMessage / addMessage	Appends a bubble or replaces a loading/error bubble; adds the mode label and copy control.
addTypingIndicator	Displays the temporary loading dots and accessible status text.
addError	Creates Retry and, when available, Read FAQ answer instead controls.
sendRequest	Sends the JSON request, applies the timeout, reads JSON and updates reply/context.
submit	Reads a question, saves a snapshot of its context, displays the user bubble and sends it.

Function(s)	Purpose
openTopic	Displays actual reviewed source questions under one of the three home topics.
loadHealth / loadTopics	Reads startup availability and suggestion data from Flask.
Event listeners below these functions	Handle Enter/Shift+Enter, click actions, optional-name dialog and New chat confirmation.

fetch() is how JavaScript calls Flask without reloading the whole page. async/await lets the browser wait for the response while staying interactive. A request state keeps the original question/context for retry. An older retry must not overwrite a newer selected-program context. structuredClone() snapshots the small request context. AbortController applies a timeout.

textContent and createTextNode display characters instead of interpreting them as HTML. The assistant supports only a small bold/list format. User messages remain literal, and unmatched asterisks remain text. Copy answer removes the supported bold markers before writing plain text to the clipboard. This is why an HTML-like answer is safe and why no general HTML renderer is needed.

8. Data, assets, settings and generated files

knowledge.json is the factual source collection. question_examples.json and question_dataset.csv are the separate learning corpus. A model label links back to a source ID; generated questions do not add new institutional facts. Each CSV row includes a split and provenance. The original source is not established as openly licensed, and its current institutional accuracy still needs department review.

static/assets contains the supplied logo/profile image, plus/send/bulb SVGs, Inter fonts and license/provenance notes. They make the UI look like the inspected Figma design. Fonts/images do not participate in model training. docs/screenshots stores real browser evidence and the named design reference; JSON files there record executed scenarios.

research/results is generated evidence: model.json is learned counts; report.json is the measured analysis; prepared_dataset.csv is cleaned data; experiments.csv is the validation comparison; test_predictions.csv contains all test predictions; figures contains the five plots. Keep these artifacts so another person can reproduce and audit the claimed results.

requirements.txt pins the runtime/test libraries. requirements-research.txt adds the optional chart and document libraries. .env.example is safe public setup text; the real .env is local and ignored. .gitignore also excludes the virtual environment and caches. .gitattributes stabilizes CSV line endings and marks document exports as binary so Git does not alter their bytes. __pycache__ and .pytest_cache are generated by Python/test execution and are not project logic.

9. Tests and document utilities

Tests use the same functions with controlled inputs and assert expected behavior. The common structure is arrange, act, assert: prepare a test app or tiny dataset, call a route/function, then check the result. Parametrized tests repeat one useful check for multiple inputs. Fixtures provide reusable setup. Mocks replace paid/external calls with deterministic responses; a mocked success does not establish live provider access.

```
# A simplified explanation of a real kind of API test:
client = create_app({"TESTING": True, "CHAT_MODE": "faq"}).test_client()
response = client.post("/api/chat", json={"message": "Who is the dean?"})
assert response.status_code == 200
assert response.get_json()["mode"] == "faq"
```

test_api.py and test_knowledge.py cover basic behavior/source selection. test_chat_improvements.py covers variations, follow-ups, unsupported details and formatting. test_ai_service.py, test_diagnostics.py and test_model_finder.py cover optional provider handling safely. test_research.py checks the Bayes hand calculation, bigrams, cleaning, split leakage, stale models, numerical reproduction and research routes. Browser scripts exercise the actual HTML/CSS/JavaScript in Chromium. evaluate_questions.py records real local FAQ responses for the older 95 development cases.

build_question_dataset.py only materializes the frozen authoring examples as CSV. build_project_documents.py reads measured artifacts and uses helper functions to build paragraphs/headings/tables/figures, then writes Word/PDF/Markdown and presentation formats. Its font/layout helpers format documents and do not change predictions. build_walkthrough.py builds this teaching document and reads Python syntax trees to produce a current function index. These offline packaging scripts are useful to explain at a high level after the app/data workflow.

10. Rebuild it through a suggested seven-day sequence

Use this as a practice/build plan. Start with a small running version each day, write the next part yourself, compare it with the full code and explain it before moving on. Record the work and contributions you actually perform. The professor permits AI assistance subject to disclosure; the papers already disclose OpenAI Codex help with code, examples, tests and drafting.

A practical sequence for three learners

Day	Build and understand	Proof you understand it
1	Create the virtual environment, install Flask, render the existing HTML/CSS and connect one simple route.	Explain request, response, render_template, static files and the page IDs.

Day	Build and understand	Proof you understand it
2	Load the FAQ JSON, write a simple lookup, then add unknown-question and privacy handling.	Trace one supported and one unsupported question; identify where facts are stored.
3	Connect browser fetch(), display safe replies, add bold text, retry and small program follow-ups.	Explain the sent JSON, context reset, loading state and safe text rendering.
4	Review the constructed CSV/schema; implement normalization, duplicate/split checks and three EDA charts.	Explain each field and why exact train/test duplication would be a problem.
5	Count class/word frequencies, implement Naive Bayes, compare the four fixed settings and evaluate once with the frozen protocol.	Work the 9/13 toy example and distinguish train, validation and test.
6	Connect the saved model to guarded ranking and build the separate analysis demo; run the tests.	Explain why a wrong/high-scoring prediction cannot establish unknown opening hours.
7	Review the generated papers/slides, verify facts/permissions, rehearse the demo and assign presentation sections.	Each member can trace a question, describe the dataset and explain one observed model error.

A small first Flask version to practice

Write this in a separate practice folder so it does not replace the full application. It teaches the first request/response step. Then add JSON loading, validation and source selection incrementally rather than copying the whole final backend at once.

```
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.post("/api/chat")
def chat():
    body = request.get_json(silent=True)
    if not isinstance(body, dict):
        return jsonify(error="Send a JSON object."), 400

    question = body.get("message", "")
    if not isinstance(question, str) or not question.strip():
        return jsonify(error="Enter a question."), 400

    return jsonify(reply="You asked: " + question)

if __name__ == "__main__":
    app.run(debug=False)
```

11. Run the real project and practice changes

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
py -m pip install -r requirements-research.txt
py analyze.py --dataset data/question_dataset.csv
py app.py
```

Use an already activated environment if you have one. For local FAQ answers, set CHAT_MODE=faq in .env beside app.py. The classifier and analysis need no provider key. Open the local server home for chat and add /analysis for the experiment demonstration. Stop an older Flask process with Ctrl+C before restarting after Python/model changes.

• In the browser Network tab, inspect the JSON for a supported question and explain every field.

• Change a harmless greeting in a practice copy and predict which file must change.

• Add a new paraphrase to a future training revision and explain why you need new independent test evidence after inspecting current errors.

• Run the test for the Bayes hand calculation and explain why its expected answer is 9/13.

• Try a missing model artifact in a practice copy and explain why ordinary source retrieval can still function.

• Inspect one actual misclassification in test_predictions.csv and propose a testable improvement without claiming it has already worked.

12. Questions you should prepare for the defense

Explain these ideas in your own words

Question	Key points for an accurate answer
What is your implemented AI algorithm?	A locally trained multinomial Naive Bayes question-to-FAQ classifier; explain counts, priors, smoothing and the independence assumption.
Is this only a Gemini API wrapper?	The classifier, dataset analysis and FAQ answers run without a provider key. Gemini/OpenAI is a separate optional wording service.
Where did the data come from?	22 supplied department FAQs plus 264 explicitly AI-assisted constructed question examples, not collected student chats.
Why not call 95/95 your model accuracy?	Those are development regression cases. The separate classifier test is 28/44, with macro F1 0.595.

Question	Key points for an accurate answer
Why keep source rules if you have ML?	Closed-set predictions can be wrong and do not prove a fact exists. Source eligibility and missing-detail checks serve a different purpose.
Why a small dataset?	It demonstrates the full course workflow, but the test is too small/constructed for deployment or population claims.
Why JSON and CSV?	A small read-only factual collection fits JSON; a row-based labeled learning dataset fits CSV. No private student-record database is part of this scope.
Why those four configurations?	A prespecified, explainable comparison of word-order features and two smoothing values; validation selects, test evaluates.
What is the biggest weakness?	Actual errors in related intents and some Filipino wordings, limited labels/language review, small constructed test set and unconfirmed current department facts.
Did you use AI assistance?	Describe Codex assistance and your own actual review, implementation, experiments and learning accurately. Demonstrate understanding by tracing code and reproducing results.
What would you improve next?	Current source confirmation, permissioned real questions, independent labels and a fresh test set; then justified algorithm and out-of-scope experiments.

13. Current Python function index

These line numbers are generated from the current source. Use your editor's Go to Line command. Leading underscores mark internal helpers by convention. The explanations above describe the logic; this index helps locate the exact function.

Runtime, research and utility functions

File	Function / starting line	Purpose or reading note
app.py	create_app (line 46)	Create the Flask app, load its files, and connect URLs to functions.
app.py	index (line 82)	See the file walkthrough and the numbered/commented block around this function.
app.py	health (line 86)	See the file walkthrough and the numbered/commented block around this function.

File	Function / starting line	Purpose or reading note
app.py	suggestions (line 115)	See the file walkthrough and the numbered/commented block around this function.
app.py	chat (line 140)	Follow a question from input checking to its final JSON answer.
app.py	http_error (line 257)	See the file walkthrough and the numbered/commented block around this function.
chatbot/retriever.py	normalize (line 69)	See the file walkthrough and the numbered/commented block around this function.
chatbot/retriever.py	_canonical_word (line 81)	See the file walkthrough and the numbered/commented block around this function.
chatbot/retriever.py	tokens (line 91)	Turn a sentence into a set of meaningful, consistently named words.
chatbot/retriever.py	program_entities (line 101)	See the file walkthrough and the numbered/commented block around this function.
chatbot/retriever.py	intents (line 105)	Identify requested topics using the documented keyword lists above.
chatbot/retriever.py	_supported_detail (line 138)	A broad topic match cannot establish a missing fee, deadline or contact.
chatbot/retriever.py	retrieve (line 173)	Filter sources first, then rank eligible records by topic and word support.
chatbot/conversation.py	read_context (line 8)	Accept only program names and an optional topic awaiting clarification.
chatbot/conversation.py	resolve_question (line 32)	Add the previous program name to a follow-up, or ask the user to choose.
chatbot/conversation.py	is_private_question (line 73)	Detect requests for personal grades/passwords; no record database exists.
chatbot/answers.py	_sentences (line 6)	See the file walkthrough and the numbered/commented block around this function.

File	Function / starting line	Purpose or reading note
chatbot/answers.py	_summary (line 10)	See the file walkthrough and the numbered/commented block around this function.
chatbot/answers.py	_emphasize (line 69)	Mark key facts already present in the extractive reply, without adding facts.
chatbot/answers.py	format_faq (line 93)	See the file walkthrough and the numbered/commented block around this function.
chatbot/answers.py	source_metadata (line 103)	Keep source review information in the API, separate from the visible reply.
chatbot/intent_model.py	clean_question (line 16)	See the file walkthrough and the numbered/commented block around this function.
chatbot/intent_model.py	features (line 21)	See the file walkthrough and the numbered/commented block around this function.
chatbot/intent_model.py	knowledge_fingerprint (line 32)	A changed source gets a different hash, so an old model can be ignored.
chatbot/intent_model.py	__init__ (line 45)	See the file walkthrough and the numbered/commented block around this function.
chatbot/intent_model.py	fit (line 90)	Learn by counting examples and words in each FAQ class.
chatbot/intent_model.py	predict (line 115)	Apply the learned counts; return labels from highest to lowest score.
chatbot/intent_model.py	save (line 150)	See the file walkthrough and the numbered/commented block around this function.
chatbot/intent_model.py	load_model (line 155)	Ignore absent/stale/invalid artifacts; keep ordinary retrieval available.
chatbot/ai_service.py	generate (line 12)	Ask the selected provider to phrase an answer using our retrieved records.
chatbot/ai_service.py	list_models (line 41)	List IDs through the configured provider; this does not generate text.

File	Function / starting line	Purpose or reading note
chatbot/errors.py	__init__ (line 14)	See the file walkthrough and the numbered/commented block around this function.
chatbot/errors.py	diagnose (line 21)	Inspect structured codes only; never return or log the exception itself.
research/dataset.py	count_values (line 16)	Count a categorical column for a table or bar chart.
research/dataset.py	load_dataset (line 24)	Read a CSV, keep valid rows, and return the observed cleaning statistics.
research/evaluation.py	classification_metrics (line 3)	See the file walkthrough and the numbered/commented block around this function.
research/evaluation.py	evaluate (line 57)	See the file walkthrough and the numbered/commented block around this function.
research/web.py	read_report (line 19)	See the file walkthrough and the numbered/commented block around this function.
research/web.py	analysis (line 31)	See the file walkthrough and the numbered/commented block around this function.
research/web.py	analysis_data (line 38)	See the file walkthrough and the numbered/commented block around this function.
research/web.py	analysis_figure (line 46)	See the file walkthrough and the numbered/commented block around this function.
research/web.py	classify (line 53)	See the file walkthrough and the numbered/commented block around this function.
analyze.py	better_configuration (line 21)	Apply the same four selection rules in an easy-to-read order.
analyze.py	save_csv (line 34)	See the file walkthrough and the numbered/commented block around this function.

File	Function / starting line	Purpose or reading note
analyze.py	figures (line 41)	See the file walkthrough and the numbered/commented block around this function.
analyze.py	finish (line 53)	See the file walkthrough and the numbered/commented block around this function.
analyze.py	run (line 99)	See the file walkthrough and the numbered/commented block around this function.
analyze.py	main (line 171)	See the file walkthrough and the numbered/commented block around this function.
check_ai.py	model_candidates (line 15)	Prefer recent stable Flash-Lite IDs, then Flash; exclude media/probes.
check_ai.py	find_working_model (line 28)	See the file walkthrough and the numbered/commented block around this function.
check_ai.py	run (line 60)	See the file walkthrough and the numbered/commented block around this function.
check_ai.py	main (line 121)	See the file walkthrough and the numbered/commented block around this function.
tools/build_question_dataset.py	main (line 10)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	para (line 17)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	head (line 21)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	bullets (line 25)	See the file walkthrough and the numbered/commented block around this function.

File	Function / starting line	Purpose or reading note
tools/build_project_documents.py	table (line 29)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	figure (line 33)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	cover (line 37)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	content (line 45)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	inline_runs (line 359)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	register_pdf_fonts (line 366)	Embed fonts so PDF viewers never substitute incompatible glyph metrics.
tools/build_project_documents.py	write_markdown (line 386)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	write_docx (line 411)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	write_pdf (line 496)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	markup (line 517)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	page (line 557)	See the file walkthrough and the numbered/commented block around this function.
tools/build_project_documents.py	slides (line 566)	See the file walkthrough and the numbered/commented block around this function.

File	Function / starting line	Purpose or reading note
tools/build_project_documents.py	main (line 640)	See the file walkthrough and the numbered/commented block around this function.
tools/build_walkthrough.py	code (line 17)	See the file walkthrough and the numbered/commented block around this function.
tools/build_walkthrough.py	role_for_file (line 82)	See the file walkthrough and the numbered/commented block around this function.
tools/build_walkthrough.py	file_inventory (line 122)	See the file walkthrough and the numbered/commented block around this function.
tools/build_walkthrough.py	function_index (line 131)	See the file walkthrough and the numbered/commented block around this function.
tools/build_walkthrough.py	guide (line 151)	See the file walkthrough and the numbered/commented block around this function.
tools/build_walkthrough.py	main (line 452)	See the file walkthrough and the numbered/commented block around this function.

14. Complete tracked/deliverable file inventory

Every included source, data, asset, test and document path is listed here. The ignored local .env, virtual environment and generated Python/test caches are intentionally outside the repository inventory; their roles were explained earlier. Generated Word/PDF/PowerPoint files are outputs, not hidden backend code.

File-by-file purpose

File	Why it exists / what it does
.env.example	Public configuration example. Copy it locally; put the real key only in the ignored .env.
.gitattributes	Keeps CSV newlines consistent and stores PDF/Word/PowerPoint documents as binary files.

File	Why it exists / what it does
.gitignore	Keeps .env, virtual-environment files, caches and logs out of Git.
README.md	Main setup, run commands, feature summary and links to the coursework materials.
analyze.py	Offline CSV-to-model experiment: cleaning, splitting, training, selection, testing and charts.
app.py	Starts Flask, loads settings/data/model and handles the main chat URLs. Read this first.
chatbot/__init__.py	Marks chatbot as a Python package; it has no answer logic.
chatbot/ai_service.py	Optional Gemini/OpenAI SDK calls for source-grounded wording and model listing.
chatbot/answers.py	Turns selected source records into short FAQ replies and bold markers; prepares hidden metadata.
chatbot/conversation.py	Validates the small program context, resolves follow-ups and detects private-record questions.
chatbot/errors.py	Converts provider failures into fixed, safe messages and troubleshooting actions.
chatbot/intent_model.py	Implements the local Naive Bayes learner and predictions using ordinary Python counts.
chatbot/retriever.py	Chooses source-supported FAQ records using word/topic rules and a small learned ranking bonus.
check_ai.py	Optional command-line check of provider configuration/model access; it hides credentials.
data/README.md	Dataset dictionary, provenance, split rationale, cleaning rules and permission status.
data/knowledge.json	The 22 department source FAQs and review metadata; these supply answer facts.
data/question_dataset.csv	The actual 264-row classification dataset consumed by analyze.py.
data/question_examples.json	Frozen AI-assisted question examples, grouped by target FAQ; not student transcripts.

File	Why it exists / what it does
docs/department-verification.md	The unresolved institutional facts and source-currency checks for department review.
docs/design-reference.json	Recorded Figma frame dimensions, node references and typography; no chat algorithm.
docs/design.md	Explains how the Figma design was inspected and translated into the preserved UI.
docs/evaluation-improved.json	Latest actual responses to the 95 development HTTP checks.
docs/evaluation.json	Earlier evaluation snapshot retained as development history; it is not the current model test.
docs/guideline-compliance.md	Maps each professor requirement to code, evidence and deliverables.
docs/improvements.md	Earlier matching, follow-up and answer-presentation changes and their evidence.
docs/learning/CCIThink_Beginner_Walkthrough.docx	This beginner learning/defense guide or its navigation page; not application runtime code.
docs/learning/CCIThink_Beginner_Walkthrough.pdf	This beginner learning/defense guide or its navigation page; not application runtime code.
docs/learning/README.md	This beginner learning/defense guide or its navigation page; not application runtime code.
docs/learning/walkthrough.md	This beginner learning/defense guide or its navigation page; not application runtime code.
docs/paper/CCIThink_Defense.pdf	Defense presentation export or speaker notes; generated from the measured result artifacts.
docs/paper/CCIThink_Defense.pptx	Defense presentation export or speaker notes; generated from the measured result artifacts.
docs/paper/CCIThink_IMRaD_Draft.docx	Requested IMRaD paper draft in the named format; requires author review.
docs/paper/CCIThink_IMRaD_Draft.pdf	Requested IMRaD paper draft in the named format; requires author review.
docs/paper/CCIThink_Technical_Report_Draft.docx	Professor-required five-chapter report draft in the named format; requires author review.

File	Why it exists / what it does
docs/paper/CCIThink_Technical_Report_Draft.pdf	Professor-required five-chapter report draft in the named format; requires author review.
docs/paper/README.md	Navigation and review instructions for the papers and presentation.
docs/paper/defense-slides.md	Defense presentation export or speaker notes; generated from the measured result artifacts.
docs/paper/imrad.md	Requested IMRaD paper draft in the named format; requires author review.
docs/paper/reference-notes.md	Citation sources, retrieval notes, confirmed authors and AI-assistance disclosure.
docs/paper/technical-report.md	Professor-required five-chapter report draft in the named format; requires author review.
docs/presentation.md	Presentation context and links to the actual defense deck and notes.
docs/project-proposal.md	Submission proposal: problem, objectives, dataset, method and expected outputs.
docs/research-guide.md	Hands-on setup, import, demonstration, experiment and reproduction instructions.
docs/screenshots/analysis/browser-results.json	Timestamped browser scenario report showing executed checks and their scope.
docs/screenshots/analysis/desktop-analysis.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/analysis/mobile-analysis.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/browser-results.json	Timestamped browser scenario report showing executed checks and their scope.
docs/screenshots/conversation.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/desktop.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/figma-reference.png	Actual browser screenshot or clearly named design reference used as visual evidence.

File	Why it exists / what it does
docs/screenshots/improvements/browser-results.json	Timestamped browser scenario report showing executed checks and their scope.
docs/screenshots/improvements/mobile-careers-followup.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/improvements/mobile-focused-answer.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/loading.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/mobile-conversation.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/mobile.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/screenshots/tablet.png	Actual browser screenshot or clearly named design reference used as visual evidence.
docs/source-review.md	Earlier transcription/source-review notes for the imported department material.
docs/testing.md	Current measured test results and the limits of what each test establishes.
requirements-research.txt	Adds plotting and Word/PDF/PowerPoint export packages for reproducible coursework artifacts.
requirements.txt	Pinned application and test packages; the local classifier uses the Python standard library.
research/__init__.py	Marks research as a Python package; it contains no training logic itself.
research/dataset.py	Loads and cleans the CSV, prevents exact split leakage and describes the actual data.
research/evaluation.py	Counts correct predictions, false positives/negatives and the confusion matrix.
research/results/experiments.csv	The four validation-only configurations and their actual metrics.
research/results/figures/class_distribution.png	A generated research chart from actual data/results; the filename identifies its subject.

File	Why it exists / what it does
research/results/figures/language_distribution.png	A generated research chart from actual data/results; the filename identifies its subject.
research/results/figures/question_lengths.png	A generated research chart from actual data/results; the filename identifies its subject.
research/results/figures/test_confusion.png	A generated research chart from actual data/results; the filename identifies its subject.
research/results/figures/validation_experiment.png	A generated research chart from actual data/results; the filename identifies its subject.
research/results/model.json	Saved trained counts, vocabulary, settings and hashes; read at startup, not retrained per message.
research/results/prepared_dataset.csv	Cleaned/normalized rows with the original split assignment retained.
research/results/report.json	Measured data profile, validation comparison, test metrics, all errors and limitations.
research/results/test_predictions.csv	Every selected-model test prediction, actual label, correctness and relative score.
research/web.py	Groups the separate analysis, figure and classifier demonstration routes.
static/assets/README.md	Supplied design image or asset provenance/license notes; no answer facts are taken from sample design text.
static/assets/ccit-logo.png	Supplied design image or asset provenance/license notes; no answer facts are taken from sample design text.
static/assets/fonts/OFL.txt	Self-hosted Inter font or its open-font license; visual appearance only.
static/assets/fonts/inter-latin-400.woff2	Self-hosted Inter font or its open-font license; visual appearance only.
static/assets/fonts/inter-latin-500.woff2	Self-hosted Inter font or its open-font license; visual appearance only.
static/assets/fonts/inter-latin-600.woff2	Self-hosted Inter font or its open-font license; visual appearance only.
static/assets/icons/bulb.svg	Original vector UI icon: plus/new chat, send or topic bulb; no Python behavior.

File	Why it exists / what it does
static/assets/icons/plus.svg	Original vector UI icon: plus/new chat, send or topic bulb; no Python behavior.
static/assets/icons/send.svg	Original vector UI icon: plus/new chat, send or topic bulb; no Python behavior.
static/assets/sample-avatar.png	Supplied design image or asset provenance/license notes; no answer facts are taken from sample design text.
static/css/analysis.css	Extra styles scoped to the analysis page so the chat design stays unchanged.
static/css/style.css	The preserved chat design: fonts, red colors, layout, messages and responsive rules.
static/js/analysis.js	Sends a demo question to the local classifier and displays prediction text safely.
static/js/script.js	Sends chat requests, displays safe formatted replies and manages tab-local UI state.
templates/analysis.html	Displays measured research results with Flask/Jinja; it is separate from the chat home.
templates/index.html	The original chat page: logo/header, greeting, composer, topic cards and dialogs.
tests/browser_analysis.py	Checks charts, local classification, mobile layout, failures and safe prediction display.
tests/browser_improvements.py	Checks concise bold replies, copy, follow-ups, retries and safe rendering.
tests/browser_smoke.py	Checks the original page and chat controls in real Chromium across six viewport sizes.
tests/evaluate_questions.py	Runs the older 95 development cases against a live local FAQ server and records results.
tests/fixtures/student_questions.json	The 95 development regression cases; they are separate from the classifier test split.
tests/test_ai_service.py	Tests real SDK request/response handling with in-memory transports; no real API key.
tests/test_api.py	Tests basic routes, invalid input, private/unknown questions and missing setup.

File	Why it exists / what it does
tests/test_chat_improvements.py	Tests wording variants, missing facts, follow-ups, fallback and bold key facts.
tests/test_diagnostics.py	Checks safe CLI output and classification of malformed provider errors.
tests/test_knowledge.py	Checks that documented questions retrieve the intended source FAQ.
tests/test_model_finder.py	Checks the optional model search cap, stop rules and preservation of private settings.
tests/test_research.py	Tests Bayes arithmetic, bigrams, cleaning, leakage rejection, model files and research routes.
tools/build_project_documents.py	Formats measured results into papers and defense slides; does not answer chat questions.
tools/build_question_dataset.py	Converts the authored question lists into a reproducible CSV.
tools/build_walkthrough.py	Builds this guide and a current source-function/file index without running the app.

15. Review checklist before presenting

• Every member can explain the numbered chat and experiment steps without memorizing a script.

• Every member can identify source facts, constructed training examples and model predictions as different things.

• The local demo starts in the chosen mode and the actual result files match the paper/slides.

• You can reproduce the toy Bayes arithmetic and explain at least one real test error.

• You have reviewed the authorship/AI-assistance disclosure and describe actual contributions accurately.

• Current department facts, source permissions and instructor-specific formatting have appropriate review.